

Guida Completa

Sviluppare CRM/ERP Funzionanti con Claude Code

Tecniche, strategie e best practices per creare sistemi enterprise
evitando errori di integrazione tra moduli

Argomento:	Architettura Software con AI
Focus:	CRM, ERP, Sistemi Enterprise
Tool:	Claude Code, LLM-assisted development
Livello:	Intermedio / Avanzato

Indice dei Contenuti

- 1. Il Problema dell'Integrazione nei Sistemi AI-Generated
- 2. Principio Fondamentale: Architettura Prima del Codice
- 3. Sistema di File Documentali (Contratti Architetturali)
- 4. Tecniche di Prompting Strategico
- 5. Struttura Tecnica Anti-Rottura (Single Source of Truth)
- 6. README per Claude Code
- 7. Vertical Slice vs Layer Approach
- 8. Template Prompt per Nuovi Moduli
- 9. Strumenti per Mantenere Coerenza (Zod, TypeScript)
- 10. Strategie di Migrazione per Codice Legacy
- 11. Workflow Pratico Completo
- 12. Il Prompt Magico (Le 5 Domande)
- Appendice: Template File Architettura

1. Il Problema dell'Integrazione

Il problema più comune nello sviluppo assistito da AI è che i **task isolati funzionano perfettamente, ma l'integrazione tra moduli fallisce**.

Sintomi classici:

- Il modulo Clienti funziona perfettamente
- Il modulo Fatture funziona perfettamente
- Collegandoli insieme: errori TypeScript, foreign key mancanti, tipi incompatibili
- Modifichi Clienti → si rompe Preventivi
- Modifichi Preventivi → si rompe Reporting
- Loop infinito di fix e regressioni

Causa radice: Gli LLM lavorano a task isolati senza visione sistemica delle dipendenze tra moduli.

Questo non è un limite dell'AI, ma della **metodologia di sviluppo**. La soluzione è fornire all'AI il contesto architetturale completo prima di scrivere codice.

2. Architettura Prima del Codice

Invece di scrivere codice e poi tentare di integrarlo, definiamo prima **l'architettura completa del sistema**, poi implementiamo moduli che la rispettano.

Workflow tradizionale (ERRATO):

1. "Claude, crea modulo gestione clienti"
2. "Claude, crea modulo fatturazione"
3. "Claude, crea modulo reporting"
4. ■ Niente si integra correttamente

Workflow corretto:

1. Definisci ARCHITECTURE.md (mappa del sistema)
2. Definisci DATA_MODEL.md (schema DB unificato)
3. Definisci API_CONTRACTS.md (interfacce tra moduli)
4. Definisci MODULE_DEPENDENCIES.md (grafo dipendenze)
5. Crea /src/types/shared/ (tipi condivisi)
6. SOLO ORA: implementa moduli seguendo i contratti
7. ■ Tutto si integra perché rispetta l'architettura

3. File Documentali (Contratti)

Prima di scrivere codice, crea questi file nella cartella /docs:

File	Scopo
ARCHITECTURE.md	Mappa sistema: moduli, flussi dati, dipendenze
DATA_MODEL.md	Schema database unificato con relazioni
API_CONTRACTS.md	Interfacce pubbliche tra moduli
MODULE_DEPENDENCIES.md	Grafo dipendenze: chi usa chi
CONVENTIONS.md	Standard codice e naming

Esempio: ARCHITECTURE.md per un CRM

```
# CRM System - Architecture

## Moduli Core

1. **Auth Module**
  - Gestione utenti e autenticazione
  - Tabelle: users, sessions
  - Exports: login(), logout(), getCurrentUser()

2. **Customers Module**
  - Gestione anagrafica clienti
  - Tabelle: customers, customer_contacts
  - Exports: createCustomer(), getCustomer(), searchCustomers()
  - Dipendenze: Auth (user context)

3. **Invoices Module**
  - Gestione fatturazione
  - Tabelle: invoices, invoice_items
  - Exports: createInvoice(), getInvoice(), listInvoices()
  - Dipendenze: Customers (customer_id), Auth (user)

4. **Reporting Module**
  - Analytics e statistiche
  - Tabelle: nessuna (solo lettura)
  - Exports: getSalesReport(), getCustomerStats()
  - Dipendenze: Customers, Invoices (aggregazione)

## Flusso Dati Principale
Auth → tutti i moduli (user context)
Customers → Invoices (customer_id reference)
Customers + Invoices → Reporting (aggregazione)

## Shared Types Location
/src/types/shared/
  - User.ts
  - Customer.ts
  - Invoice.ts
  - common.ts
```

4. Tecniche di Prompting Strategico

La formulazione del prompt determina se otterrai codice isolato o integrato.

■ PROMPT SBAGLIATO (microtask):

"Crea modulo fatturazione con CRUD fatture"

Risultato: codice senza considerare:

- Tipi esistenti nel progetto
- Collegamento con altri moduli
- Convenzioni da seguire
- Foreign keys necessarie

■ PROMPT CORRETTO (macrotask contestuale):

"Prima leggi /docs/ARCHITECTURE.md e /docs/DATA_MODEL.md.

Crea il modulo Invoices che:

1. ANALISI
 - Identifica moduli da cui dipende (Customers, Auth)
 - Verifica tipi condivisi esistenti
 - Leggi convenzioni in CONVENTIONS.md
2. IMPLEMENTAZIONE
 - Usa tipi da /src/types/shared/Invoice.ts
 - Importa Customer da shared per customer_id
 - Rispetta naming conventions
 - Espone API secondo API_CONTRACTS.md
 - Gestisci autenticazione con Auth module
3. VERIFICA
 - Tipi TypeScript matchano con Customers
 - Foreign keys esistono nel DB
 - Modulo importabile da Reporting
 - Nessun breaking change
4. DOCUMENTAZIONE
 - Aggiorna API_CONTRACTS.md
 - Aggiorna MODULE_DEPENDENCIES.md

Conferma step-by-step prima di procedere."

Risultato: codice perfettamente integrato

5. Single Source of Truth per i Tipi

Centralizza tutti i tipi condivisi per eliminare duplicazioni.

Struttura directory:

```
/src
  /types
    /shared          ← SINGLE SOURCE OF TRUTH
      index.ts       ← Tutti i tipi pubblici
      User.ts
      Customer.ts
      Invoice.ts
      common.ts      ← ID, Status, Timestamp
    /modules
      /auth
        service.ts
        routes.ts
        types.ts      ← Solo tipi LOCALI
      /customers
        service.ts
        routes.ts
        types.ts
      /invoices
        service.ts
        routes.ts
        types.ts
```

Esempio `/src/types/shared/index.ts`:

```
// SINGLE SOURCE OF TRUTH
export interface User {
  id: string;
  email: string;
  role: 'admin' | 'user' | 'viewer';
  created_at: Date;
}

export interface Customer {
  id: string;
  name: string;
  email: string;
  phone?: string;
  created_by: string; // FK to users
  created_at: Date;
}

export interface Invoice {
  id: string;
  customer_id: string; // FK to customers
  invoice_number: string;
  amount: number;
  status: InvoiceStatus;
  issued_date: Date;
  created_by: string; // FK to users
}

export type InvoiceStatus = 'draft' | 'sent' | 'paid' | 'overdue';
```

Come i moduli usano questi tipi:

```
// src/modules/customers/service.ts
import { Customer, User } from '@types/shared';

export class CustomerService {
  async createCustomer(
    data: Omit<Customer, 'id' | 'created_at'>,
  ) {
```

```
        user: User
      ): Promise<Customer> {
        // Implementazione
      }
    }

// src/modules/invoices/service.ts
import { Invoice, Customer, User } from '@types/shared';

export class InvoiceService {
  async createInvoice(
    customerId: string, // Type-safe da Customer.id
    data: Omit<Invoice, 'id' | 'customer_id'>,
    user: User
  ): Promise<Invoice> {
    // Claude sa che customerId deve esistere!
  }
}
```


6. README per Claude Code

Crea README_CLAUDE.md nella root: il manuale operativo per Claude.

```
# Istruzioni per Claude Code

## LEGGI SEMPRE PRIMA DI OGNI TASK

### Pre-requisiti
1. /docs/ARCHITECTURE.md - contesto sistema
2. /docs/DATA_MODEL.md - se tocchi DB
3. /src/types/shared/index.ts - tipi esistenti
4. /docs/API_CONTRACTS.md - interfacce pubbliche

### Workflow Obbligatorio

**STEP 1: ANALISI**
- Identifica moduli impattati
- Verifica tipi condivisi esistenti
- Controlla dipendenze in MODULE_DEPENDENCIES.md

**STEP 2: DESIGN**
- Nuovi tipi condivisi → /src/types/shared/
- Nuove API → documenta in API_CONTRACTS.md
- Modifiche dipendenze → aggiorna MODULE_DEPENDENCIES.md

**STEP 3: IMPLEMENTAZIONE**
- Scrivi codice
- Usa tipi da shared/
- Rispetta CONVENTIONS.md
- Non duplicare logica

**STEP 4: VERIFICA**
- TypeScript compila senza errori
- Test passano
- Moduli dipendenti non rotti
- Documenta breaking changes

**STEP 5: DOCUMENTAZIONE**
- Aggiorna file /docs se necessario

### Convenzioni

**Naming:**
- camelCase: variabili/funzioni
- PascalCase: tipi/interfacce
- kebab-case: file
- snake_case: database

**Import corretto:**
```typescript
// ■ CORRETTO
import { Customer } from '@types/shared';

// ■ SBAGLIATO
interface Customer { ... } // se esiste in shared
```

### Testing Integrazione
Dopo modifiche verificare:
- npm run type-check → zero errori
- Import da moduli dipendenti funzionano
- Foreign keys DB esistono
```

7. Vertical Slice vs Layer Approach

Sviluppare a 'strati' porta a scoprire problemi troppo tardi.

■ Layer Approach (SBAGLIATO):

Fase 1: Tutto il DB
Fase 2: Tutti i services
Fase 3: Tutte le routes
Fase 4: ■ Niente funziona insieme

■ Vertical Slice (CORRETTO):

Iteration 1: Auth COMPLETO (DB + service + routes)
■ Login/logout testato e funzionante

Iteration 2: Customers COMPLETO
■ CRUD clienti + integrazione Auth

Iteration 3: Invoices COMPLETO
■ Fatture + integrazione Customers + Auth

Iteration 4: Reporting COMPLETO
■ Sistema completo funzionante

Principio: ogni slice è deployable e testabile

8. Template Prompt per Nuovi Moduli

Aggiungi modulo [NOME] al sistema.

STEP 1: ANALISI

- Leggi /docs/ARCHITECTURE.md
- Leggi /src/types/shared/index.ts
- Dimmi:
 - * Moduli esistenti impattati
 - * Tipi riusabili
 - * Dipendenze circolari da evitare

STEP 2: DESIGN

Proponi:

- Tipi da aggiungere a shared/
- Schema DB con FK
- API pubbliche
- Dipendenze

ATTENDI APPROVAZIONE

STEP 3: IMPLEMENTAZIONE

- Implementa rispettando design approvato
- Aggiorna documentazione

STEP 4: VERIFICA

- TypeScript compila
- Moduli dipendenti OK
- Test passano
- Documenta breaking changes

9. Strumenti per Coerenza

Zod: validazione runtime che matcha TypeScript

```
// src/types/shared/schemas.ts
import { z } from 'zod';

export const CustomerSchema = z.object({
  id: z.string().uuid(),
  name: z.string().min(1),
  email: z.string().email(),
  phone: z.string().optional(),
  created_by: z.string().uuid(),
  created_at: z.date(),
});

export type Customer = z.infer<typeof CustomerSchema>;

// Uso nei service
const validData = CustomerSchema.parse(input);
// Se passa, validData è garantito Customer valido
```

Altri tool:

1. TypeScript strict mode (tsconfig.json)
2. ESLint import rules (forza uso shared/)
3. Madge per dipendenze circolari
4. SQL migration tool (Prisma, Knex)

10. Migrazione Codice Esistente

Strategia: Strangler Fig Pattern (migrazione graduale)

Fase 1: FOTOGRAFIA

"Claude, analizza codice e crea:

- ARCHITECTURE_CURRENT.md (stato attuale)
- TYPES_AUDIT.md (tipi duplicati)
- DEPENDENCIES_GRAPH.md (grafo dipendenze)"

Fase 2: DESIGN TARGET

"Proponi:

- Architettura pulita target
- Migration path prioritizzato
- Breaking changes previsti"

Fase 3: REFACTORING INCREMENTALE

Modulo per modulo:

1. Crea versione nuova in /modules/
2. Usa tipi da shared/
3. Mantieni API compatibility
4. Testa
5. Elimina codice vecchio
6. Next modulo

Sistema sempre funzionante durante migrazione

11. Workflow Pratico Completo

SESSIONE 1: Fondamenta (1-2 ore)

- Analisi stato attuale
- Design architettura target
- Creazione file /docs

SESSIONE 2: Tipi (30-60 min)

- Centralizzazione in /src/types/shared/
- Setup Zod schemas

SESSIONI 3+: Moduli (1-2 ore/modulo)

Per ogni modulo in ordine dipendenze:

- Refactoring
- Testing integrazione
- Approvazione

SESSIONE FINALE: Cleanup

- Elimina codice vecchio
- Verifica type-check
- Test integrazione completi

MANUTENZIONE CONTINUA

- Ogni feature: usa template Step 1-4
- Ogni 3-6 mesi: review architettura

12. Il Prompt Magico

Un singolo prompt che previene il 90% degli errori di integrazione:

Prima di scrivere codice, rispondi:

1. Quali file LEGGERAI per contestualizzare?
2. Quali file MODIFICHERAI o CREERAI?
3. Quali tipi/interfacce USERAI da shared?
4. Quali moduli potrebbero ROMPERSI?
5. Come VERIFICHERAI che tutto funziona?

Aspetto risposta prima di procedere.

Perché funziona:

- ✓ Forza pensiero sistemico
- ✓ Rivela dipendenze nascoste
- ✓ Permette correzioni preventive
- ✓ Crea checklist testing
- ✓ Previene 90% errori integrazione

Usa questo prompt **sempre**, anche per task apparentemente semplici.

Appendice: Template File

Template ARCHITECTURE.md

```
# [Sistema] - Architecture

## Moduli Core

### 1. [ModuleName]
**Responsabilità:** [descrizione]
**Tabelle:** [lista]
**API:** [funzioni pubbliche]
**Dipendenze:** [altri moduli]

## Data Flow
[Diagramma flussi critici]

## Shared Types
/src/types/shared/

## Conventions
Vedere CONVENTIONS.md
```

Template DATA_MODEL.md

```
# Data Model

## ERD
[Diagramma relazioni]

## Tables

### table_name
```sql
CREATE TABLE table_name (
 id UUID PRIMARY KEY,
 -- colonne
 FOREIGN KEY (ref) REFERENCES other(id)
);
```

**Relationships:** [descrizione]
```

Template API_CONTRACTS.md

```
# API Contracts

## [ModuleName]

```typescript
interface ModuleService {
 method(params): Promise<Return>;
}
```

**Validazione:** [regole]
**Errori:** [tipi errore]
**Autorizzazione:** [policy]
```


Conclusioni

Principi chiave da ricordare:

- 1. Architettura PRIMA del codice
- 2. Single source of truth per tipi
- 3. Vertical slice, non layer approach
- 4. Usa sempre il prompt delle 5 domande
- 5. Valida runtime con Zod
- 6. README_CLAUDE.md come riferimento
- 7. Mai saltare fase di analisi
- 8. Migrazione graduale, non rewrite
- 9. Test integrazione dopo ogni modulo
- 10. Mantieni documentazione aggiornata

Seguendo questa metodologia, trasformerai Claude Code da generatore di codice isolato in **architetto di sistemi integrati e funzionanti**.

La differenza tra successo e fallimento sta nel **pensare al sistema completo prima di scrivere la prima riga di codice**.